Seminary #1

The member/2 predicate

```
member(X, L):- L=[H|T], X=H.
member(X, L):- L=[H|T], X\=H, member(X, T).
member(X, []):- fail.
```

The member/2 predicate

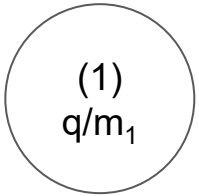
```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]          member(X, T).

[q] ?- member(2, [1,2,3]).
```

- A deduction tree (contains only non-erased nodes) becomes an *execution tree* when finished (*contains all nodes even those erased - by failure+backtrack*).

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate [someone to the blackboard]



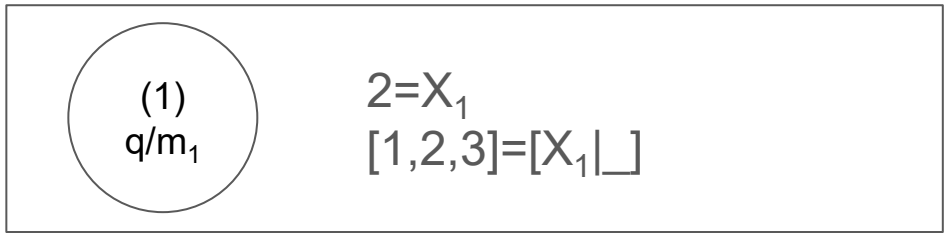

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```

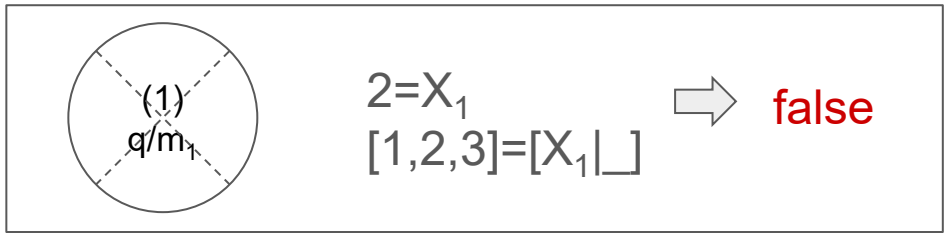


R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```



C1 ✗

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

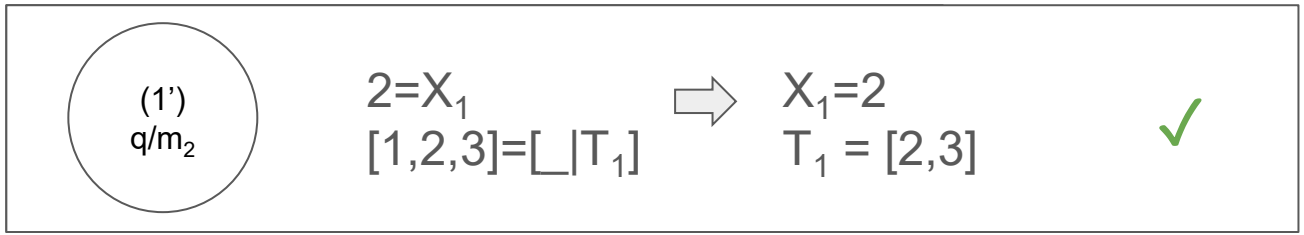
```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```



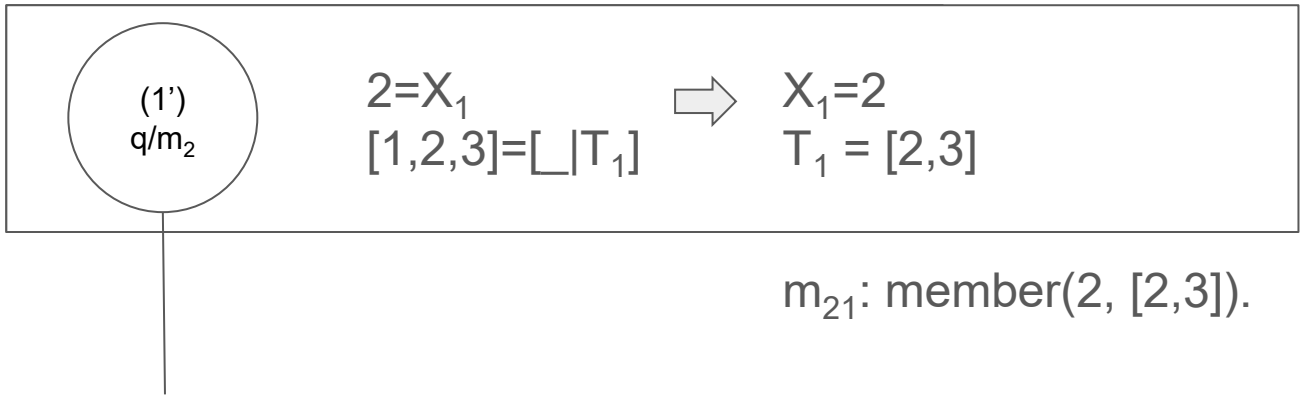
C1 ✓
C2 ✗

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```

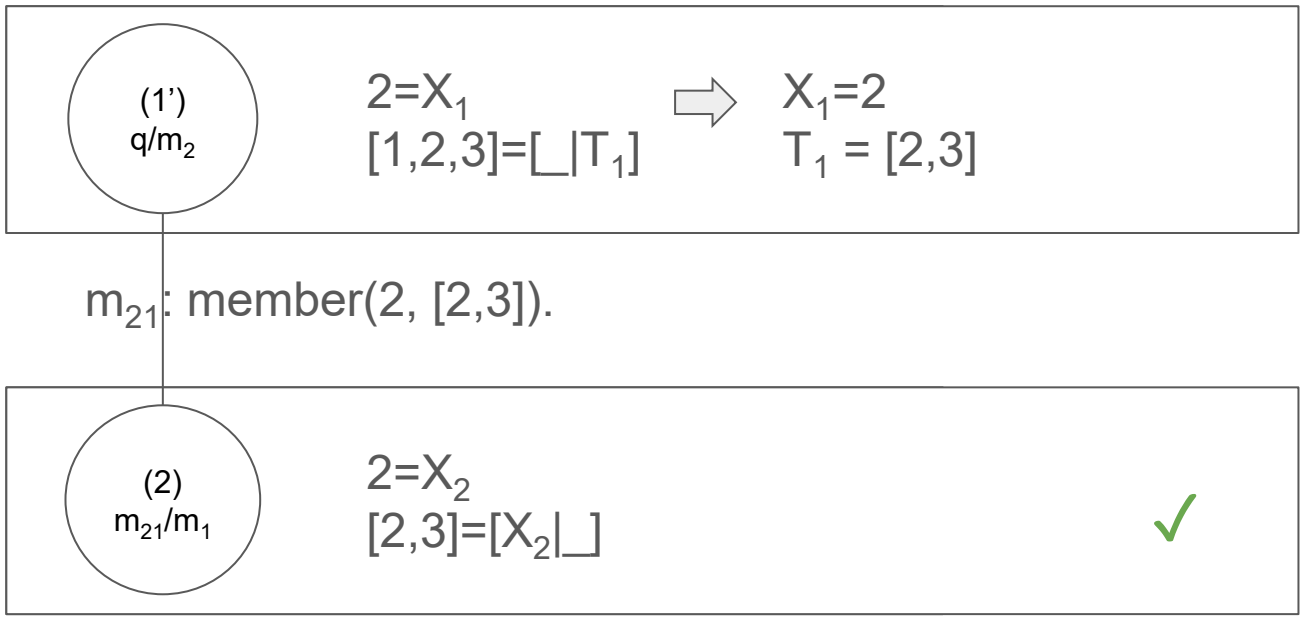


R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```

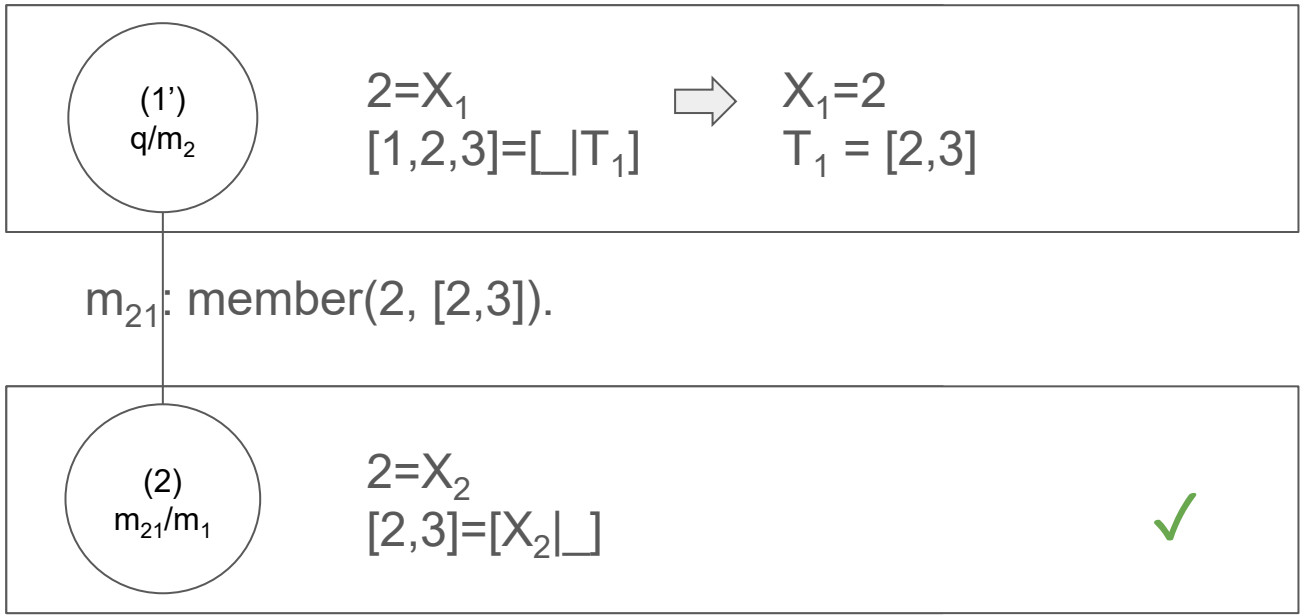


R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```



C1 ✓
C2 ✗

C1 ✓
C2 ✓
C3 ✓

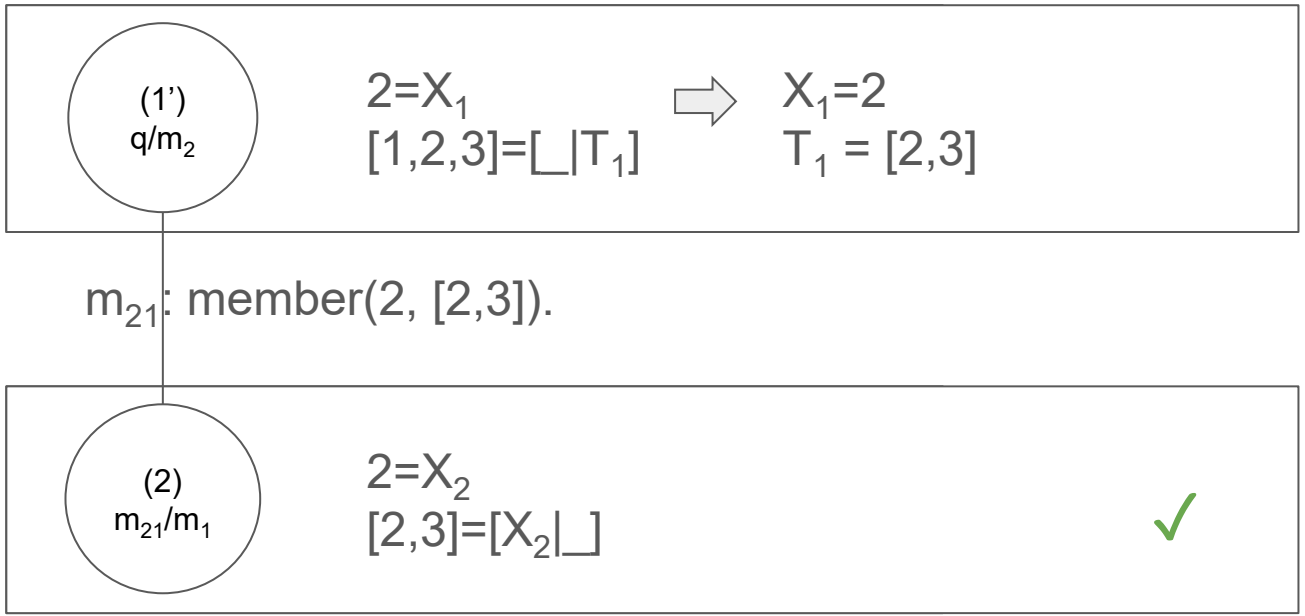
R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

We have overlapped with a fact, the node is a leaf, and we do not have any other clause in the stack. We have obtained the execution tree

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

The member/2 predicate

```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(2, [1,2,3]).
```



Do remember:

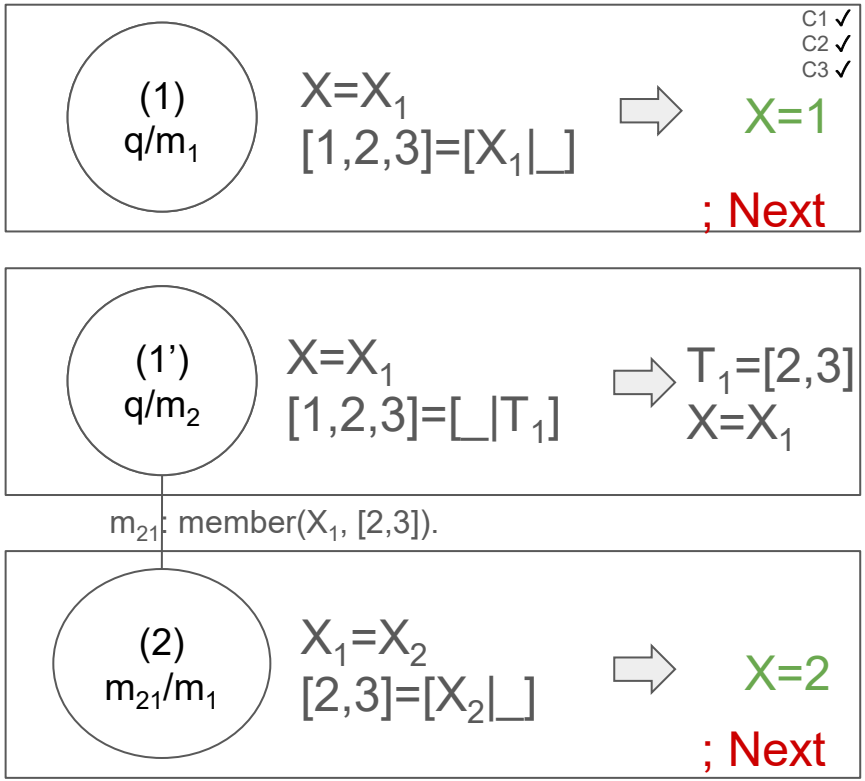
- (1) failed and only then through (1') did we get an answer.

R1: clauses → top-down
R2: subgoals → left-right
R3: failure → backtrack

C1: q/head succeeds? →C2 or backtrack
C2: fact? → C3 or body push stack
C3: Stack empty? Stop or pop

We have overlapped with a fact, the node is a leaf, and we do not have any other clause in the stack. We have obtained the execution tree

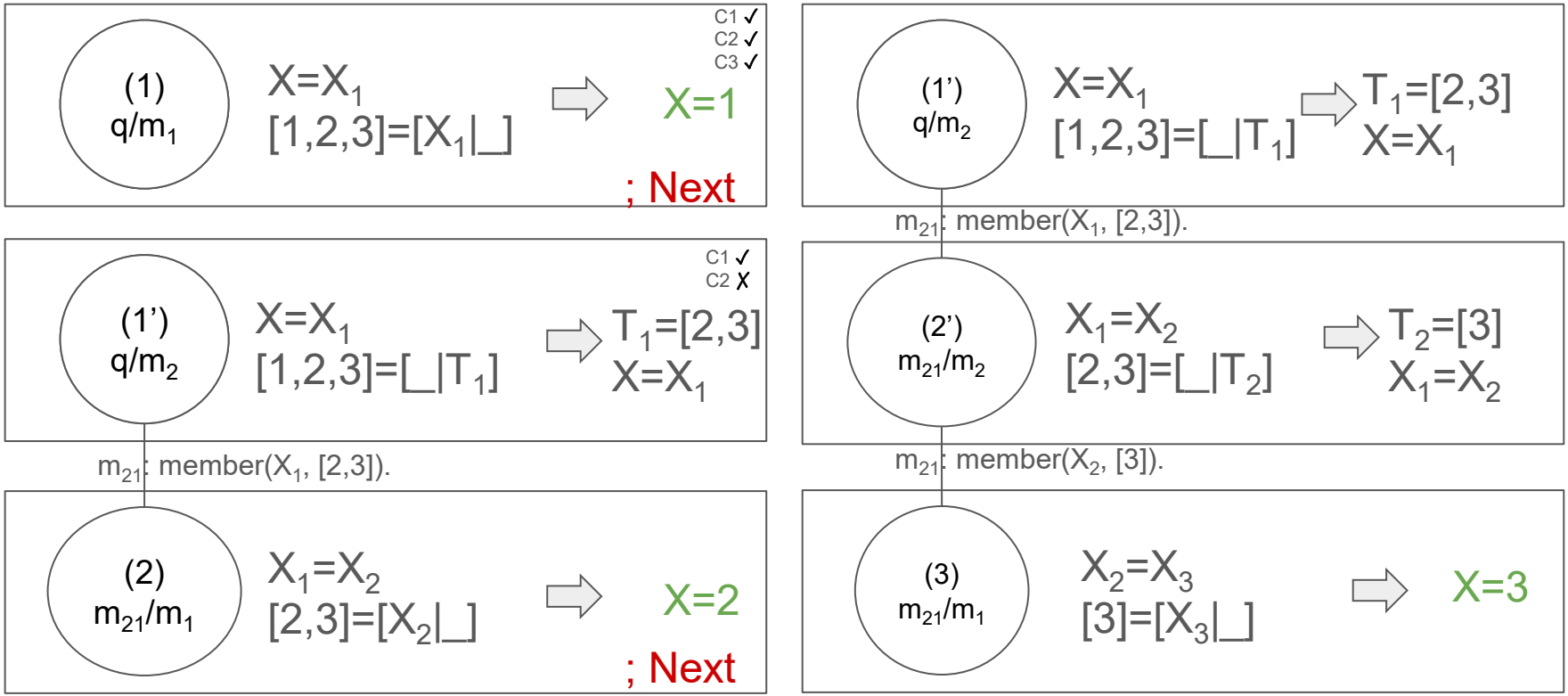
The member/2 predicate




```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]       member(X, T).

[q] ?- member(X, [1,2,3]).
```

The member/2 predicate

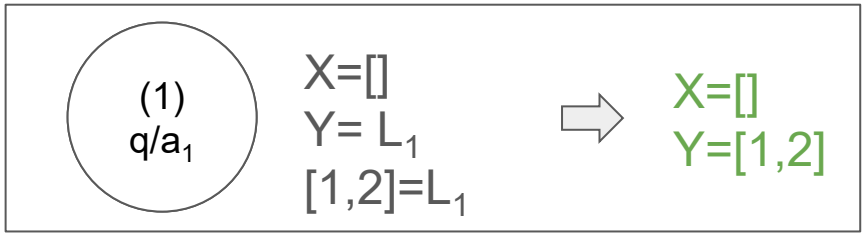
```
[m1] member(X, [X|_]).
[m2] member(X, [_|T]):-
[m21]      member(X, T).

[q] ?- member(X, [1,2,3]).
```



The append/3 predicate

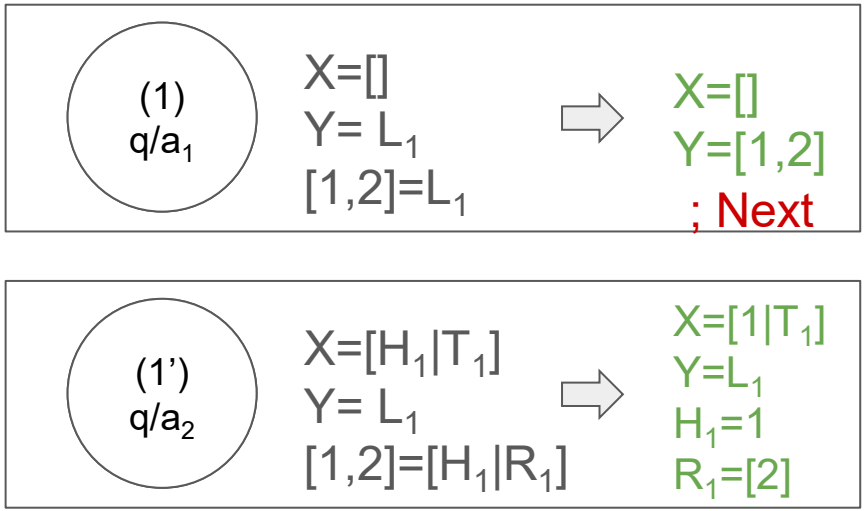
```
[a1] append([], L, L).
[a2] append([H|T], L, [H|R]):-
[a21]          append(T, L, R).

[q] ?- append(X, Y, [1,2]).
```

The append/3 predicate




```
[a1] append([], L, L).
[a2] append([H|T], L, [H|R]):-
[a21]      append(T, L, R).

[q] ?- append(X, Y, [1,2]).
```

The append/3 predicate

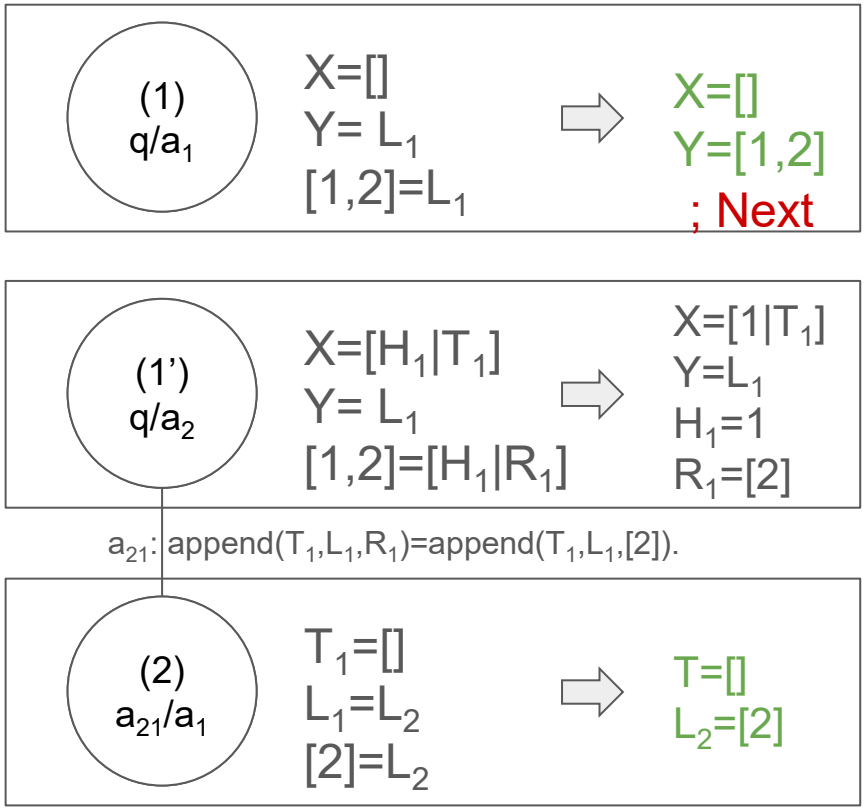


a_{21}: append(T_1,L_1,R_1)=append(T_1,L_1,[2]).

```
[a1] append([], L, L).
[a2] append([H|T], L, [H|R]):-
[a21]      append(T, L, R).

[q] ?- append(X, Y, [1,2]).
```

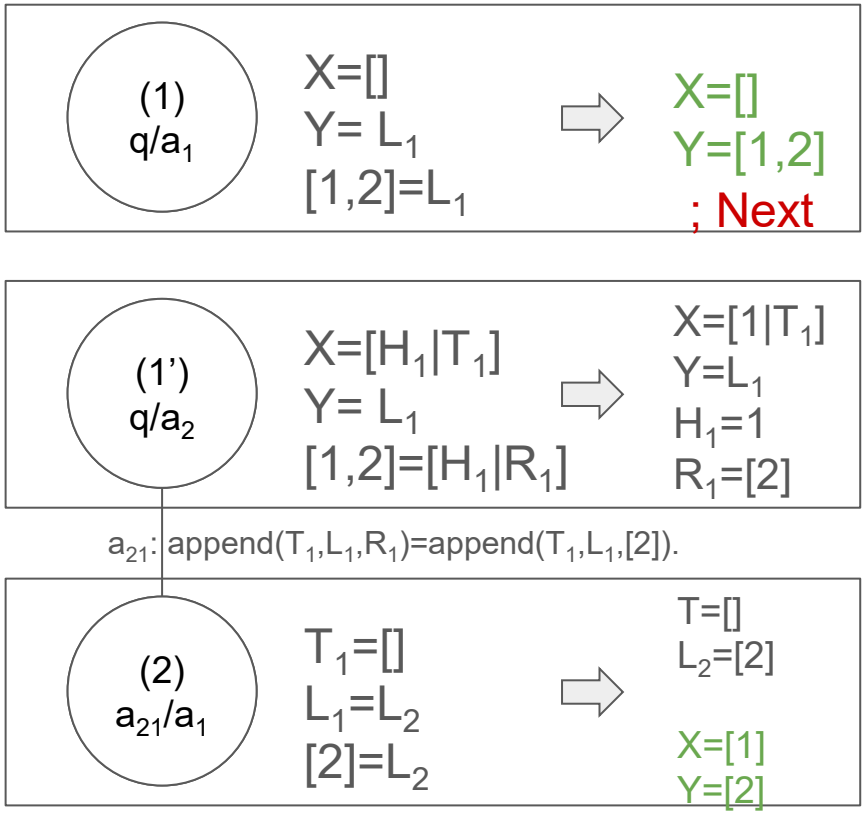
The append/3 predicate

```
[a1] append([], L, L).
[a2] append([H|T], L, [H|R]):-
[a21]       append(T, L, R).

[q] ?- append(X, Y, [1,2]).
```



The append/3 predicate

```
[a1] append([], L, L).
[a2] append([H|T], L, [H|R]):-
[a21]      append(T, L, R).

[q] ?- append(X, Y, [1,2]).
```

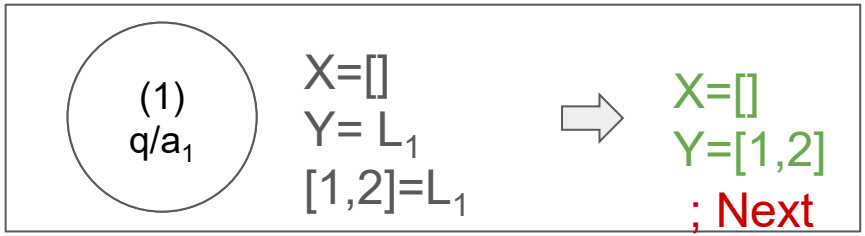


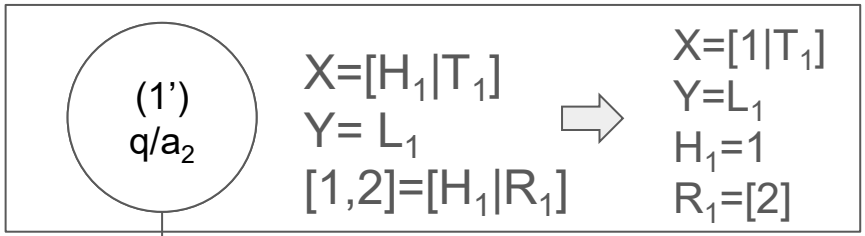
$$X \overset{(1')}{==} [1|T_1] \overset{(2)}{==} [1]$$

$$Y \overset{(1')}{==} L_1 \overset{(2)}{==} [2]$$

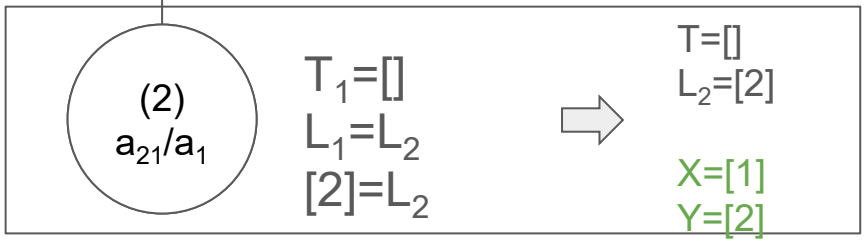
The append/3 predicate

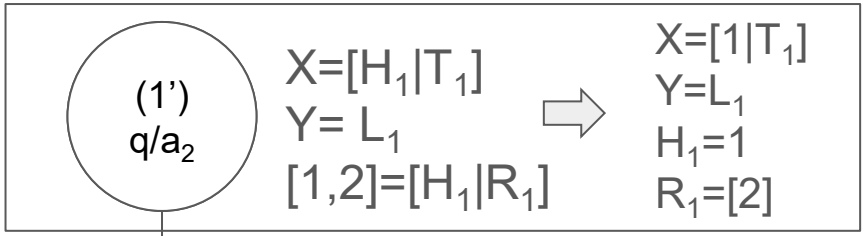
```
[a1] append([], L, L).
[a2] append([H|T], L, [H|R]):-
[a21]      append(T, L, R).

[q] ?- append(X, Y, [1,2]).
```

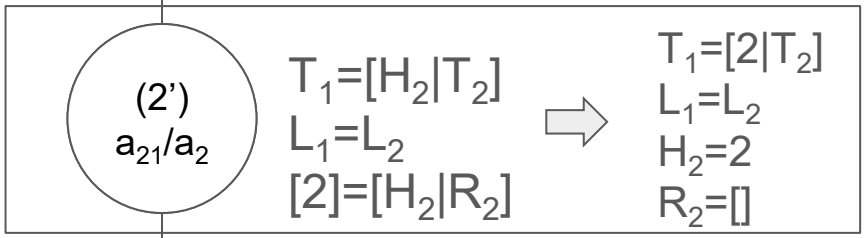




a_{21}: append(T_1,L_1, R_1)=append(T_1,L_1,[2]).







a_{21}: append(T_2,L_2,[]).